34-pwa-offline

34. PWA & Offline-First

Level: Intermediate
Jam: 8 (4 minggu × 2 sesi)
Prasyarat: JavaScript/TypeScript, Web Basics (HTML, CSS, Fetch API)
Output: Progressive Web App dengan dukungan offline + push notifications

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Membuat Service Worker dari nol — lifecycle install, activate, fetch
- Mengelola cache dengan Cache API & strategi caching (stale-while-revalidate, cache-first, network-first)
- Menggunakan IndexedDB untuk penyimpanan data offline dengan library idb
- Sinkronisasi data offline → online pakai Background Sync API
- Membuat & memasang Web App Manifest (name, icons, theme_color, display)
- Memasang PWA ke layar utama via beforeinstallprompt
- Mengirim & menerima push notifications dengan VAPID + web-push
- Melakukan audit PWA dengan Lighthouse
- Memahami offline-first architecture & conflict resolution

Materi

Sesi	Topik	File
1	Service Worker — lifecycle, cache strategies, Workbox, offline fallback	01-service-worker.md
2	IndexedDB — offline storage, Background Sync, conflict resolution	02-indexeddb-offline.md
3	Web App Manifest — manifest.json, install prompt, splash screen, Lighthouse audit	03-pwa-manifest.md
4	Push Notifications — Push API, VAPID keys, web-push, notification actions	04-push-notifications.md

Output Akhir Modul

> **PWA Notes App** — Aplikasi catatan progresif yang: - Bisa diinstall ke layar utama (manifest + beforeinstallprompt) - Berfungsi offline penuh (Service Worker + Cache API) - Menyimpan data di IndexedDB & sync saat online - Mengirim reminder via push notification - Skor Lighthouse ≥ 90 untuk PWA

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- “Explain this Service Worker lifecycle step by step”
- “Find the bug — my SW cache strategy serves stale data”
- “Generate VAPID keys and show me the secure setup”
- “Refactor this IndexedDB code to use the idb library”
- “Simulate a conflict scenario in offline-first sync and resolve it”
- “Audit this PWA manifest against the latest Lighthouse criteria”
- “Generate a push notification payload that handles click actions”
- “Explain why my beforeinstallprompt doesn’t fire”

1. Service Worker

Konsep Dasar

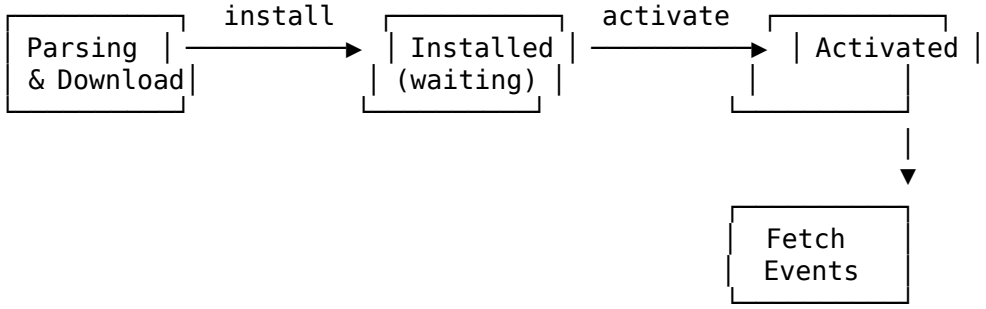
Service Worker (SW) adalah script JavaScript yang jalan di background browser, terpisah dari halaman web. SW bisa:

- **Mengintercept request** — nangkap fetch request dan decide mau ambil dari cache atau jaringan
- **Mengelola cache** — nyimpen resource (HTML, CSS, JS, gambar) biar bisa akses offline
- **Push notification** — nerima pesan dari server meskipun tab ditutup
- **Background sync** — nunda action sampe koneksi balik

Perbedaan dengan Web Worker

Aspek	Web Worker	Service Worker
Tujuan	Compute berat di background	Proxy jaringan, cache, offline
Akses DOM	□	□
Akses Cache API	□	□
Lifecycle	Langsung jalan & mati	install → activate → fetch
Persistensi	Ikut halaman	Independent dari halaman
Bisa intercept fetch	□	□

Lifecycle Service Worker



Event: `install`

Dipanggil sekali pas SW pertama kali di-download. Biasanya buat **pre-cache** resource awal.

```
// sw.js
const CACHE_NAME = 'notes-app-v1';
const PRECACHE_URLS = [
  '/',
  '/index.html',
  '/style.css',
  '/app.js',
  '/offline.html'
];

self.addEventListener('install', event => {
  console.log('[SW] Install');

  // WaitUntil — browser nunggu promise selesai
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => {
      console.log('[SW] Pre-caching resources');
      return cache.addAll(PRECACHE_URLS);
    })
  );

  // Langsung activate, tanpa nunggu tab lain ditutup
  self.skipWaiting();
});
```

Event: `activate`

Dipanggil setelah install, pas SW siap ngambil alih kendali. Biasanya buat **bersihin cache lama**.

```
self.addEventListener('activate', event => {
  console.log('[SW] Activate');

  const cacheAllowlist = [CACHE_NAME];

  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames.filter(name => !cacheAllowlist.includes(name))
          .map(name => caches.delete(name))
      );
    })
```

```
  );

  // Langsung kontrol halaman tanpa refresh
  self.clients.claim();
});
```

Event: `fetch`

Dipanggil setiap kali halaman bikin request (HTML, API, gambar dll).
Ini jantungnya offline support.

```
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(cached => {
      // Fallback: coba jaringan dulu, kalo gagal pake cache
      return fetch(event.request)
        .then(response => {
          // Update cache dengan response baru
          caches.open(CACHE_NAME).then(cache => {
            cache.put(event.request, response.clone());
          });
          return response;
        })
        .catch(() => cached || caches.match('/offline.html'));
    })
  );
});
```

Mendaftarkan Service Worker dari Halaman

```
// main.js — di halaman web utama
if ('serviceWorker' in navigator) {
  window.addEventListener('load', async () => {
    try {
      const registration = await navigator.serviceWorker.register('/sw.js', {
        scope: '/'  // SW hanya kontrol path dalam scope ini
      });
      console.log('[SW] Registered:', registration.scope);

      // Update otomatis pas SW baru terdeteksi
      registration.addEventListener('updatefound', () => {
        console.log('[SW] Update found — installing...');
      });
    } catch (err) {
      console.error('[SW] Registration failed:', err);
    }
```

```
  });
}
```

Cek Status SW di Console

```
// Dev tools utility
navigator.serviceWorker.getRegistrations().then(regs => {
  console.table(regs.map(r => ({
    scope: r.scope,
    state: r.active?.state ?? 'none',
    installing: r.installing?.state ?? '-',
    waiting:   r.waiting?.state ?? '-'
  })));
});
```

Cache Strategies

1. Stale-While-Revalidate (default — paling aman)

Kirim **cache dulu**, update cache dari jaringan di background.

```
// Cepat — cocok buat asset yang jarang berubah
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(cached => {
      const fetchPromise = fetch(event.request).then(response => {
        caches.open(CACHE_NAME).then(cache => cache.put(event.request, response)
        return response.clone();
      }).catch(() => cached);

      return cached || fetchPromise;
    })
  );
});
```

Contoh use case: Avatar user, logo, CSS framework.

2. Cache-First

Cek cache dulu. Kalo ada → pake. Kalo gak ada → fetch dari jaringan & cache.

```
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(cached => {
      if (cached) return cached;
```

```
      return fetch(event.request).then(response => {
        caches.open(CACHE_NAME).then(cache => cache.put(event.request, response)
        return response.clone();
      });
    })
  );
});
```

Contoh use case: Gambar static, font, halaman yang jarang berubah.

3. Network-First

Coba jaringan dulu. Kalo gagal → ambil dari cache (fallback offline).

```
self.addEventListener('fetch', event => {
  event.respondWith(
    fetch(event.request)
      .then(response => {
        caches.open(CACHE_NAME).then(cache => cache.put(event.request, response)
        return response.clone();
      })
      .catch(() => caches.match(event.request))
  );
});
```

Contoh use case: API response, data yang sering berubah.

4. Cache-Only

Cuma pake cache. Gak nyentuh jaringan sama sekali.

```
self.addEventListener('fetch', event => {
  event.respondWith(caches.match(event.request));
});
```

Contoh use case: Halaman offline, asset yang gak pernah berubah.

Matriks Pemilihan Strategi

Strategi	Kecepatan	Kesegaran Data	Reliability
Cache-Only	⚡ Sangat cepat	□ Basi	□ Tinggi
Cache-First	⚡ Cepat	⚠ Agak basi	□ Tinggi
Stale-While-Revalidate	⚡ Cepat	□ Segar	□ Tinggi

Strategi	Kecepatan	Kesegaran Data	Reliability
Network-First	Lambat (kalo online)	Paling segar	⚠ Rendah kalo offline

Workbox

Workbox adalah library Google yang nge-simplify Service Worker. Gak perlu nulis caching logic manual.

Setup Workbox CLI

```
npm install workbox-webpack-plugin --save-dev
npm install workbox-precaching workbox-routing workbox-strategies
```

Workbox Config (webpack)

```
// webpack.config.js
const { GenerateSW } = require('workbox-webpack-plugin');

module.exports = {
  plugins: [
    new GenerateSW({
      clientsClaim: true,
      skipWaiting: true,
      runtimeCaching: [
        {
          urlPattern: /\.(?:png|jpg|jpeg|svg|gif)$/,
          handler: 'CacheFirst',
          options: {
            cacheName: 'images',
            expiration: { maxEntries: 50, maxAgeSeconds: 30 * 24 * 60 * 60 }
          }
        },
        {
          urlPattern: /^https:\/\/api\./,
          handler: 'NetworkFirst',
          options: {
            cacheName: 'api-responses',
            networkTimeoutSeconds: 3
          }
        }
      ]
    })
```

```
  ]
};
```

Workbox Manual (tanpa build tool)

```
// sw.js — pakai importScripts
importScripts('https://storage.googleapis.com/workbox-cdn/releases/6.5.4/workbox

if (workbox) {
  console.log('[Workbox] Loaded!');

  // Precaching — otomatis dari manifest
  workbox.precaching.precacheAndRoute(self.__WB_MANIFEST || []);

  // Runtime caching
  workbox.routing.registerRoute(
    /\.(?:js|css)$/,
    new workbox.strategies.StaleWhileRevalidate({
      cacheName: 'static-assets'
    })
  );

  workbox.routing.registerRoute(
    /\.(?:png|jpg|jpeg|svg|gif|webp)$/,
    new workbox.strategies.CacheFirst({
      cacheName: 'images',
      plugins: [
        new workbox.expiration.ExpirationPlugin({
          maxEntries: 60,
          maxAgeSeconds: 30 * 24 * 60 * 60 // 30 hari
        })
      ]
    })
  );

  workbox.routing.registerRoute(
    /^https:\/\/api\./,
    new workbox.strategies.NetworkFirst({
      cacheName: 'api',
      networkTimeoutSeconds: 3
    })
  );
}
```

Offline Fallback Page

Buat halaman khusus yang tampil pas user offline.

```
<!-- offline.html -->
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Offline</title>
  <style>
    body {
      font-family: system-ui, sans-serif;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
      background: #f5f5f5;
    }
    .card {
      text-align: center;
      padding: 2rem;
      background: white;
      border-radius: 16px;
      box-shadow: 0 4px 24px rgba(0,0,0,0.1);
    }
    .icon { font-size: 4rem; }
    h1 { color: #333; }
    p { color: #666; }
    button {
      margin-top: 1rem;
      padding: 0.75rem 2rem;
      background: #007aff;
      color: white;
      border: none;
      border-radius: 8px;
      font-size: 1rem;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <div class="card">
    <div class="icon">□</div>
```

```
    <h1>Kamu Lagi Offline</h1>
    <p>Koneksi internet terputus. Coba lagi nanti.</p>
    <button onclick="window.location.reload()">Coba Lagi</button>
  </div>
</body>
</html>
```

SW routing ke offline fallback

```
self.addEventListener('fetch', event => {
  event.respondWith(
    fetch(event.request)
      .then(response => {
        // Simpan response ke cache
        caches.open(CACHE_NAME).then(cache => cache.put(event.request, response.
        return response;
      })
      .catch(() => {
        // Cek apakah request navigasi (halaman HTML)
        if (event.request.mode === 'navigate') {
          return caches.match('/offline.html');
        }
        // Coba cari di cache, kalo gak ada ya offline fallback
        return caches.match(event.request)
          .then(cached => cached || new Response('Offline', { status: 503 }));
      })
  );
});
```

Cache-First dengan Offline Fallback (lengkap)

```
const CACHE_NAME = 'pwa-notes-v1';
const PRECACHE = ['/', '/index.html', '/offline.html', '/style.css', '/app.js'];

self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => cache.addAll(PRECACHE))
  );
  self.skipWaiting();
});

self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(keys =>
      Promise.all(keys.filter(k => k !== CACHE_NAME).map(k => caches.delete(k)))
```

```
    )
  );
  self.clients.claim();
});

self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request)
      .then(cached => {
        const fetchAndCache = fetch(event.request)
          .then(res => {
            caches.open(CACHE_NAME).then(cache => cache.put(event.request, res.c
            return res;
          })
          .catch(() => cached);

        // Kalo request navigasi → ada fallback khusus
        if (event.request.mode === 'navigate' && !cached) {
          return fetchAndCache.catch(() => caches.match('/offline.html'));
        }

        return cached || fetchAndCache;
      })
  );
});
```

Latihan

1. **Register & verify SW** — Buat file `sw.js`, register dari halaman, log registration ke console. Verifikasi di DevTools > Application > Service Workers.
2. **Implementasi Stale-While-Revalidate** — Tulis handler fetch yang kirim cache dulu, update di background. Test dengan mematikan jaringan — konten lama masih muncul, pas online balik otomatis refresh.
3. **Offline fallback page** — Buat halaman `offline.html` keren dengan animasi, config SW untuk nampilin itu pas navigasi offline. Pastikan tombol “Coba Lagi” jalan.
4. **Workbox integration** — Instal workbox via npm, config runtime caching untuk images (CacheFirst) dan API (NetworkFirst dengan timeout 3 detik). Generate SW otomatis, verify di Lighthouse.

2. IndexedDB & Offline-First

Kenapa IndexedDB?

Cache API bagus buat nyimpen file (HTML, CSS, JS, gambar). Tapi kalo butuh **nyimpen data terstruktur** (catatan, user data, transactions) — pake **IndexedDB**.

Perbandingan Storage di Browser

Storage	Tipe Data	Kapasitas	Query	Async	PWA cocok?
localStorage	String key-value	~10MB	❌	❌	❌ (sync, blocking)
SessionStorage	String key-value	~10MB	❌	❌	❌
Cache API	Request/Response	Banyak	❌	✅	Untuk file
IndexedDB	Object (apa aja)	Hampir unlimited	✅ (indexes)	✅	**Untuk data**
Origin Private File System	File	Banyak	❌	✅	File besar

Kelebihan IndexedDB

- Nyimpen **object JavaScript langsung** (gak perlu JSON.stringify)
- **Indexed queries** — cari data berdasarkan field tertentu
- **Transaction-based** — atomic, bisa rollback
- **Large capacity** — ratusan MB sampai GB
- **Async** — gak blocking main thread

Konsep IndexedDB

IndexedDB punya struktur hirarkis:

```
Database
  └── Object Store (mirip table di SQL)
       ├── Record 1 (key: 1, value: { title: "Catatan 1" })
       ├── Record 2 (key: 2, value: { title: "Catatan 2" })
       └── Index (mirip index di SQL)
             └── by title, by date, etc.
```

API Native (raw) — paham konsep dulu

```
// Buka / buat database
const request = indexedDB.open('NotesDB', 1);

// Upgrade — dipanggil pas pertama kali atau version berubah
request.onupgradeneeded = event => {
  const db = event.target.result;

  // Buat object store dengan auto-increment key
  const store = db.createObjectStore('notes', {
    keyPath: 'id',
    autoIncrement: true
  });

  // Buat index untuk query
  store.createIndex('by_title', 'title', { unique: false });
  store.createIndex('by_created', 'createdAt', { unique: false });

  console.log('[IDB] Database upgraded');
};

request.onsuccess = event => {
  const db = event.target.result;
  console.log('[IDB] Database opened:', db.name, 'v' + db.version);
};

request.onerror = event => {
  console.error('[IDB] Error:', event.target.error);
};
```

CRUD dengan Native API

```
// CREATE
function addNote(db, note) {
  const tx = db.transaction('notes', 'readwrite');
  const store = tx.objectStore('notes');
  const request = store.add({
    title: note.title,
    body: note.body,
    createdAt: new Date().toISOString(),
    synced: false
  });

  request.onsuccess = () => console.log('[IDB] Note added, id:', request.result)
  request.onerror = err => console.error('[IDB] Add error:', err);
```

```
  // Tunggu transaction selesai
  return new Promise((resolve, reject) => {
    tx.oncomplete = () => resolve(request.result);
    tx.onerror = () => reject(tx.error);
  });
}

// READ — by key
function getNote(db, id) {
  const tx = db.transaction('notes', 'readonly');
  const store = tx.objectStore('notes');
  const request = store.get(id);

  return new Promise((resolve, reject) => {
    request.onsuccess = () => resolve(request.result);
    request.onerror = () => reject(request.error);
  });
}

// READ — all with index
function getNotesByTitle(db, searchTerm) {
  const tx = db.transaction('notes', 'readonly');
  const store = tx.objectStore('notes');
  const index = store.index('by_title');

  // Range query: title mulai dari searchTerm
  const range = IDBKeyRange.bound(searchTerm, searchTerm + '\uffff');
  const request = index.openCursor(range);
  const results = [];

  return new Promise((resolve, reject) => {
    request.onsuccess = event => {
      const cursor = event.target.result;
      if (cursor) {
        results.push(cursor.value);
        cursor.continue();
      } else {
        resolve(results);
      }
    };
    request.onerror = () => reject(request.error);
  });
}

// UPDATE
```

```
function updateNote(db, id, updates) {
  const tx = db.transaction('notes', 'readwrite');
  const store = tx.objectStore('notes');

  // Get dulu, merge, terus put
  store.get(id).onsuccess = event => {
    const note = event.target.result;
    const updated = { ...note, ...updates, updatedAt: new Date().toISOString() }
    store.put(updated);
  };

  return new Promise((resolve, reject) => {
    tx.oncomplete = () => resolve();
    tx.onerror = () => reject(tx.error);
  });
}

// DELETE
function deleteNote(db, id) {
  const tx = db.transaction('notes', 'readwrite');
  const store = tx.objectStore('notes');
  store.delete(id);

  return new Promise((resolve, reject) => {
    tx.oncomplete = () => resolve();
    tx.onerror = () => reject(tx.error);
  });
}
```

Library `idb` (Recommended)

API native IndexedDB **ribet** — banyak boilerplate, event-based, gak pake Promise. **Lebih pake `idb`** — wrapper kecil yang pake Promise.

Instalasi

```
npm install idb
# atau CDN
# import { openDB } from 'https://cdn.jsdelivr.net/npm/idb@7/+esm'
```

Setup dengan idb

```
import { openDB } from 'idb';

const DB_NAME = 'NotesDB';
```

```
const DB_VERSION = 1;

const dbPromise = openDB(DB_NAME, DB_VERSION, {
  upgrade(db, oldVersion, newVersion, transaction) {
    // Hapus store lama kalo ada perubahan schema
    if (oldVersion < 1) {
      const store = db.createObjectStore('notes', {
        keyPath: 'id',
        autoIncrement: true
      });
      store.createIndex('by_title', 'title');
      store.createIndex('by_created', 'createdAt');
      store.createIndex('by_synced', 'synced');
      store.createIndex('by_updated', 'updatedAt');
    }
  }
});
```

CRUD dengan idb (jauh lebih simple)

```
// CREATE
async function addNote(note) {
  const db = await dbPromise;
  const id = await db.add('notes', {
    title: note.title,
    body: note.body,
    createdAt: new Date().toISOString(),
    updatedAt: new Date().toISOString(),
    synced: false
  });
  console.log('[idb] Note added:', id);
  return id;
}

// READ — by key
async function getNote(id) {
  const db = await dbPromise;
  return db.get('notes', id);
}

// READ — semua notes, sorted by date
async function getAllNotes() {
  const db = await dbPromise;
  const index = db.transaction('notes').store.index('by_created');
  return index.getAll(); // DESC pake .reverse()
```

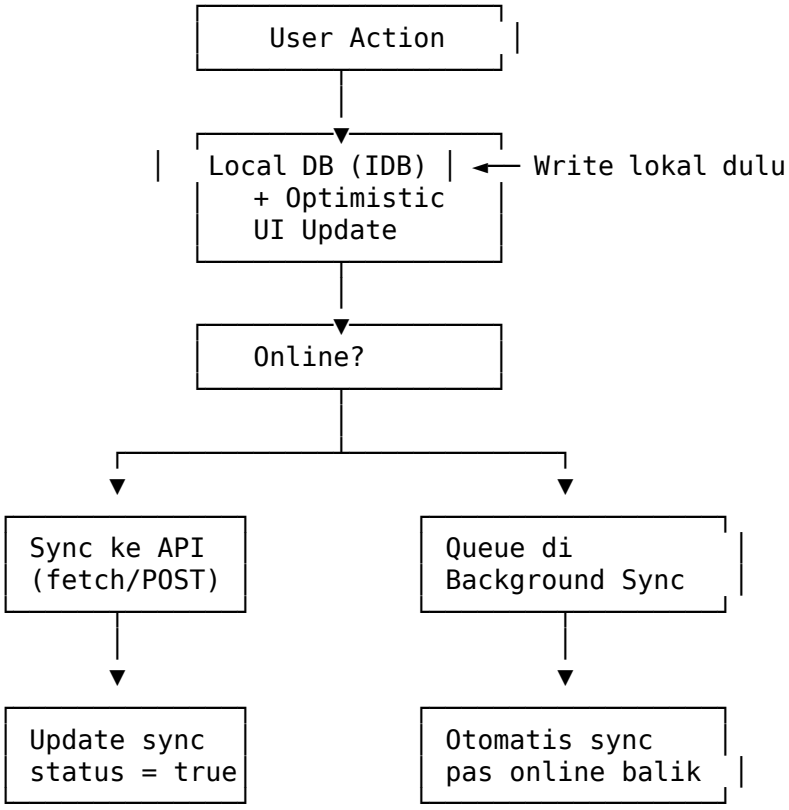
```
}

// READ — query by index
async function searchNotes(searchTerm) {
  const db = await dbPromise;
  const index = db.transaction('notes').store.index('by_title');

  // IDBKeyRange — cari yang mulai dari searchTerm
  const range = IDBKeyRange.bound(
    searchTerm,
    searchTerm + '\uffff'
  );
  return index.getAll(range);
}

// UPDATE
async function updateNote(id, updates) {
  const db = await dbPromise;
  const note = await db.get('notes', id);
  if (!note) throw new Error('Note not found');

  const updated = {
    ...note,
    ...updates,
    updatedAt: new Date().toISOString()
  };
  await db.put('notes', updated);
  return updated;
}

// DELETE
async function deleteNote(id) {
  const db = await dbPromise;
  await db.delete('notes', id);
}

// BULK — add many at once
async function addNotesBulk(notes) {
  const db = await dbPromise;
  const tx = db.transaction('notes', 'readwrite');
  const ids = await Promise.all(
    notes.map(note => tx.store.add({
      ...note,
      createdAt: new Date().toISOString(),
      synced: false
    }))
```

```
  );
  await tx.done;
  return ids;
}
```

Offline-First Architecture

Konsep

Offline-first artinya **aplikasi harus jalan dulu baru butuh internet** — bukan sebaliknya.



Optimistic UI

Rule #1 Offline-First: Update UI duluan, sync ke server belakangan.

```
// ❌ BURUK — nunggu server dulu baru update UI
async function saveNoteBad(title, body) {
  const response = await fetch('/api/notes', {
```

```
    method: 'POST',
    body: JSON.stringify({ title, body })
  });
  const saved = await response.json();
  renderedList.addNote(saved); // UI nunggu server
}

// ✅ BAIK — update UI langsung, sync belakangan
async function saveNoteGood(title, body) {
  const localNote = {
    id: `temp_${Date.now()}`,
    title,
    body,
    createdAt: new Date().toISOString(),
    synced: false
  };

  // Simpan ke IndexedDB dulu
  await addNote(localNote);

  // Update UI langsung (optimistic)
  renderedList.addNote(localNote);

  // Coba sync ke server (gagal gapapa — nanti di-retry)
  syncNoteToServer(localNote).catch(() => {
    console.log('[Sync] Gagal — akan di-retry nanti');
  });
}

async function syncNoteToServer(note) {
  const response = await fetch('/api/notes', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      title: note.title,
      body: note.body,
      clientId: note.id
    })
  });

  if (!response.ok) throw new Error('Sync failed');

  const serverNote = await response.json();

  // Update local note dengan ID server
  await updateNote(note.id, {
```

```
    id: serverNote.id,
    synced: true
  });
}
```

Background Sync API

Background Sync nunda action sampe koneksi internet balik — bahkan kalo user udah nutup tab.

Register Sync di SW

```
// Di halaman utama
async function registerSync() {
  const registration = await navigator.serviceWorker.ready;

  try {
    await registration.sync.register('sync-notes');
    console.log('[Sync] Registered background sync');
  } catch (err) {
    console.log('[Sync] Background Sync not supported:', err.message);
  }
}
```

Handle Sync Event di SW

```
// sw.js
self.addEventListener('sync', event => {
  if (event.tag === 'sync-notes') {
    console.log('[SW Sync] Triggered');
    event.waitUntil(syncUnsyncedNotes());
  }
});

async function syncUnsyncedNotes() {
  // Ambil semua data yang belum sync dari IDB
  // (pake idb import atau postMessage)

  // Kirim message ke halaman terbuka buat minta data
  const clients = await self.clients.matchAll();
  clients.forEach(client => {
    client.postMessage({ type: 'GET_UNSYNCED_NOTES' });
  });
}
```

Periodik Background Sync (opsional)

```
// Cuma support di Chrome ≥ 80 (HTTPS + PWA installed)
const status = await navigator.permissions.query({
  name: 'periodic-background-sync'
});

if (status.state === 'granted') {
  const registration = await navigator.serviceWorker.ready;
  await registration.periodicSync.register('periodic-sync', {
    minInterval: 24 * 60 * 60 * 1000 // sekali sehari
  });
}
```

Conflict Resolution

Pas online balik, data lokal dan server mungkin beda. Butuh strategi conflict resolution.

Strategi Conflict Resolution

Strategi	Cara	Cocok untuk
Last-Write-Wins (LWW)	Timestamp terakhir yang menang	Catatan sederhana
Manual Merge	Tampilin dua versi, user milih	Dokumen, edit konten
CRDT	Merge otomatis tanpa conflict	Collaborative editing
Version Vector	Tracking perubahan per node	Multi-device sync

Implementasi LWW

```
async function syncNotes() {
  const db = await dbPromise;
  const localNotes = await db.getAll('notes');

  for (const local of localNotes) {
```

```
    const response = await fetch(`/api/notes/${local.id}`, {
      headers: { 'If-Unmodified-Since': local.updatedAt }
    });

    if (response.status === 409) {
      // Conflict — server punya versi lebih baru
      const serverNote = await response.json();

      // LWW: timestamps
      if (serverNote.updatedAt > local.updatedAt) {
        // Server menang — update local
        await db.put('notes', { ...serverNote, synced: true });
      } else {
        // Local menang — overwrite server
        await fetch(`/api/notes/${local.id}`, {
          method: 'PUT',
          body: JSON.stringify(local)
        });
      }
    }
  }
}
```

Manual Merge (better UX)

```
async function handleConflict(serverNote, localNote) {
  // Kirim dua versi ke UI buat dipilih user
  const choice = await showMergeDialog({
    local: localNote,
    server: serverNote
  });

  if (choice === 'local') {
    // Pakai versi lokal
    await fetch(`/api/notes/${localNote.id}`, {
      method: 'PUT',
      body: JSON.stringify(localNote)
    });
  } else if (choice === 'server') {
    // Pakai versi server
    const db = await dbPromise;
    await db.put('notes', { ...serverNote, synced: true });
  } else if (choice === 'merge') {
    // Merge manual — user edit sendiri
    const merged = mergeNotes(serverNote, localNote);
```

```
    await fetch(`/api/notes/${merged.id}`, {
      method: 'PUT',
      body: JSON.stringify(merged)
    });
  }
}
```

Network-Aware Helper

```
function isOnline() {
  return navigator.onLine;
}

// Listen perubahan koneksi
window.addEventListener('online', () => {
  console.log('[Network] Online — trigger sync');
  syncNotes();
});

window.addEventListener('offline', () => {
  console.log('[Network] Offline — semua perubahan lokal');
  showOfflineBanner();
});

// Detection lebih reliable — ping server
async function checkServerReachable() {
  try {
    const response = await fetch('/api/health', { method: 'HEAD' });
    return response.ok;
  } catch {
    return false;
  }
}
```

Full Offline-First Flow

```
// notes-app.js — implementasi lengkap
import { openDB } from 'idb';

class NotesApp {
  constructor() {
    this.dbPromise = this.initDB();
    this.setupListeners();
  }
```

```
async initDB() {
  return openDB('NotesDB', 1, {
    upgrade(db) {
      const store = db.createObjectStore('notes', {
        keyPath: 'id',
        autoIncrement: true
      });
      store.createIndex('by_synced', 'synced');
      store.createIndex('by_updated', 'updatedAt');
      store.createIndex('by_deleted', 'deleted');
    }
  });
}

setupListeners() {
  window.addEventListener('online', () => this.syncAll());
  window.addEventListener('offline', () =>
    document.body.classList.add('offline')
  );

  // Coba sync pas halaman pertama kali load
  window.addEventListener('load', () => {
    if (navigator.onLine) this.syncAll();
  });
}

async addNote(title, body) {
  const db = await this.dbPromise;
  const note = {
    title,
    body,
    createdAt: new Date().toISOString(),
    updatedAt: new Date().toISOString(),
    synced: false,
    deleted: false
  };

  const id = await db.add('notes', note);
  this.renderNote({ ...note, id });
  this.trySync();
}

async trySync() {
  if (!navigator.onLine) return;

  try {
```

```
    await this.syncAll();
  } catch {
    // Register background sync buat retry
    const registration = await navigator.serviceWorker.ready;
    await registration.sync.register('sync-notes');
  }
}

async syncAll() {
  const db = await this.dbPromise;
  const unsynced = await db.getAllFromIndex('notes', 'by_synced', false);

  for (const note of unsynced) {
    try {
      const response = await fetch('/api/notes' + (note.deleted ? `/${note.id}
        method: note.deleted ? 'DELETE' : 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: note.deleted ? undefined : JSON.stringify(note)
      });

      if (response.ok) {
        const updated = await db.get('notes', note.id);
        updated.synced = true;

        if (note.deleted) {
          await db.delete('notes', note.id);
        } else {
          await db.put('notes', updated);
        }
      } else if (response.status === 409) {
        await this.resolveConflict(note, await response.json());
      }
    } catch (err) {
      console.log('[Sync] Failed for note', note.id, err);
    }
  }
}

renderNote(note) {
  // UI rendering logic
  console.log('[UI] Rendering note:', note.title);
}

showOfflineBanner() {
  // Tampilkan banner "Kamu offline"
}
```

```
  async resolveConflict(local, server) {
    // LWW strategy — yang paling baru menang
    if (server.updatedAt > local.updatedAt) {
      const db = await this.dbPromise;
      await db.put('notes', { ...server, synced: true });
    } else {
      await fetch(`/api/notes/${local.id}`, {
        method: 'PUT',
        body: JSON.stringify(local)
      });
    }
  }
}
```

Latihan

1. **Setup IndexedDB dengan idb** — Inisialisasi database `NotesDB` dengan object store `notes`, index `by_created` dan `by_synced`. Tulis fungsi addNote dan getAllNotes. Verifikasi data persist setelah refresh.
2. **Optimistic CRUD** — Bikin form tambah catatan yang langsung update UI tanpa nunggu server. Simpan ke IndexedDB dulu, baru coba POST ke `/api/notes`. Kalo gagal, catat di queue untuk sync nanti.
3. **Background Sync** — Integrasi Background Sync API. Pas user offline, register sync tag `sync-notes`. Di Service Worker, handle event sync dengan fetch semua unsynced data dari IndexedDB dan kirim ke server. Test dengan matikan jaringan, buat catatan, hidupkan jaringan — catatan harus otomatis tersync.
4. **Conflict Resolution UI** — Simulasi conflict: buka dua tab dengan data berbeda, sync dua arah. Implementasi LWW + modal merge. Tampilkan dua versi catatan dan kasih user pilihan: "Pakai versi lokal", "Pakai versi server", atau "Gabung manual".

3. Web App Manifest & Install Prompt

Apa Itu Web App Manifest?

Manifest adalah file JSON yang ngasih tau browser tentang aplikasi web kamu. Isinya: nama, icon, warna tema, orientasi layar, dll. Tanpa manifest, browser gak tau kalo web kamu adalah **aplikasi**.

Kenapa Manifest Penting?

Fitur	Tanpa Manifest	Dengan Manifest
Install ke home screen	□	□
Nama aplikasi di launcher	□ (pake domain)	□
Icon di launcher	□ (screenshot)	□ Custom icon
Splash screen	□ (blank putih)	□ Branded
Full screen mode	□	□ (standalone)
Status bar theme	□	□ (theme_color)
Orientation lock	□	□
App shortcuts	□	□
Share target	□	□

Struktur manifest.json

```
{
  "name": "Notes App",
  "short_name": "Notes",
  "description": "Aplikasi catatan offline-first dengan PWA",
  "start_url": "/",
  "scope": "/",
  "display": "standalone",
  "orientation": "portrait-primary",
  "theme_color": "#007aff",
  "background_color": "#ffffff",
  "lang": "id-ID",
  "dir": "ltr",
  "categories": ["productivity", "utilities"],
  "icons": [
    {
      "src": "/icons/icon-72x72.png",
      "sizes": "72x72",
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "/icons/icon-96x96.png",
      "sizes": "96x96",
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "/icons/icon-128x128.png",
      "sizes": "128x128",
```

```
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "/icons/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "/icons/icon-384x384.png",
      "sizes": "384x384",
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "/icons/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ],
  "screenshots": [
    {
      "src": "/screenshots/home.png",
      "sizes": "1080x1920",
      "type": "image/png",
      "form_factor": "narrow",
      "label": "Halaman utama Notes App"
    },
    {
      "src": "/screenshots/desktop.png",
      "sizes": "1920x1080",
      "type": "image/png",
      "form_factor": "wide",
      "label": "Tampilan desktop Notes App"
    }
  ],
  "shortcuts": [
    {
      "name": "Tambah Catatan Baru",
      "short_name": "Tambah",
      "description": "Buat catatan baru",
      "url": "/new-note",
      "icons": [{ "src": "/icons/shortcut-add.png", "sizes": "96x96" }]
    },
```

```
    {
      "name": "Catatan Terbaru",
      "short_name": "Terbaru",
      "description": "Lihat catatan terbaru",
      "url": "/recent",
      "icons": [{ "src": "/icons/shortcut-recent.png", "sizes": "96x96" }]
    }
  ],
  "related_applications": [
    {
      "platform": "play",
      "url": "https://play.google.com/store/apps/details?id=com.example.notes"
    }
  ],
  "prefer_related_applications": false,
  "display_override": ["window-controls-overlay", "standalone", "minimal-ui"],
  "handle_links": "preferred",
  "launch_handler": {
    "client_mode": "focus-existing"
  },
  "edge_side_panel": {
    "preferred_width": 480
  }
}
```

Penjelasan Key Properties

Key	Wajib	Fungsi
`name`	✅	Nama aplikasi yang muncul di launcher & splash screen
`short_name`	✅	Nama pendek (di icon, limited space)
`start_url`	✅	Halaman pertama pas aplikasi dibuka
`display`	✅	Mode tampilan: `fullscreen`, `standalone`, `minimal-ui`, `browser`
`icons`	✅	Array icon dengan berbagai ukuran
`scope`	✅	Batas URL yang dianggap bagian dari aplikasi
`theme_color`	✅	Warna status bar & toolbar

Key	Wajib	Fungsi
`background_color`	✅	Warna background splash screen
`orientation`	❌	Lock orientasi layar
`shortcuts`	❌	Context menu items di icon
`screenshots`	❌	Screenshot buat Play Store / app catalog
`categories`	❌	Kategori aplikasi
`description`	❌	Deskripsi (buat store listing)

Nilai `display`

```
{
  "display": "standalone",
  "display_override": ["window-controls-overlay", "standalone"]
}
```

Mode	Title Bar	URL Bar	Tab Bar	Cocok untuk
`fullscreen`	❌	❌	❌	Game, media player
`standalone`	✅ (custom)	❌	❌	**Kebanyakan PWA**
`minimal-ui`	✅	✅ (minimal)	✅	Utility apps
`browser`	✅	✅	✅	Fallback

Generate Icon PWA

Tools

- **PWABuilder** — https://www.pwabuilder.com/imageGenerator
- **Real Favicon Generator** — https://realfavicongenerator.net/
- **Maskable.app** — https://maskable.app/

Script Generate Icon (Node.js)

```
npm install sharp --save-dev
```

```
// scripts/generate-icons.js
const sharp = require('sharp');
const fs = require('fs');
const path = require('path');

const SIZES = [72, 96, 128, 192, 384, 512];
```

```
const INPUT = path.join(__dirname, '../public/logo-1024.png');
const OUTPUT_DIR = path.join(__dirname, '../public/icons');

if (!fs.existsSync(OUTPUT_DIR)) {
  fs.mkdirSync(OUTPUT_DIR, { recursive: true });
}

async function generateIcons() {
  for (const size of SIZES) {
    await sharp(INPUT)
      .resize(size, size)
      .toFile(path.join(OUTPUT_DIR, `icon-${size}x${size}.png`));

    // Generate maskable version (10% padding)
    const maskableSize = Math.round(size * 0.8);
    const padding = Math.round(size * 0.1);
    await sharp(INPUT)
      .resize(maskableSize, maskableSize)
      .extend({
        top: padding,
        bottom: padding,
        left: padding,
        right: padding,
        background: { r: 255, g: 255, b: 255, alpha: 1 }
      })
      .toFile(path.join(OUTPUT_DIR, `icon-${size}x${size}-maskable.png`));
  }
  console.log(`[Icons] Generated ${SIZES.length * 2} icons`);
}

generateIcons().catch(console.error);
```

Link manifest di HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Notes App</title>

  <!-- Manifest -->
  <link rel="manifest" href="/manifest.json">

  <!-- Apple touch icon (iOS fallback) -->
```

```
  <link rel="apple-touch-icon" href="/icons/icon-192x192.png">

  <!-- Meta theme color -->
  <meta name="theme-color" content="#007aff">

  <!-- iOS splash screen meta -->
  <meta name="apple-mobile-web-app-capable" content="yes">
  <meta name="apple-mobile-web-app-status-bar-style" content="black-translucent"
  <meta name="apple-mobile-web-app-title" content="Notes">

  <!-- Twitter Card -->
  <meta name="twitter:card" content="app">
  <meta name="twitter:app:name:iphone" content="Notes App">

  <!-- Facebook / Open Graph -->
  <meta property="og:title" content="Notes App">
  <meta property="og:description" content="Aplikasi catatan offline-first">
  <meta property="og:type" content="website">
</head>
```

Install Prompt (beforeinstallprompt)

Cara Kerja

Browser otomatis nampilin install banner kalo: 1. ✅ Manifest valid (name, short_name, icons ≥ 192x192, display ≠ browser) 2. ✅ Service Worker terdaftar dan aktif 3. ✅ HTTPS (atau localhost) 4. ✅ User interaksi minimal 30 detik 5. ✅ Minimal 1 halaman dikunjungi

Custom Install Button

```
// main.js
let deferredPrompt = null;
const installButton = document.getElementById('install-button');

window.addEventListener('beforeinstallprompt', event => {
  // Cegah banner otomatis
  event.preventDefault();

  // Simpan event buat trigger nanti
  deferredPrompt = event;

  // Tampilkan tombol install kustom
  installButton.style.display = 'block';
  console.log('[Install] beforeinstallprompt fired — ready to install');
```

```
  // Analytics
  gtag('event', 'beforeinstallprompt', {
    platforms: event.platforms // ['web', 'play', 'windows', 'mac']
  });
});

installButton.addEventListener('click', async () => {
  if (!deferredPrompt) return;

  // Tampilkan prompt install
  deferredPrompt.prompt();

  // Tunggu hasil user
  const { outcome } = await deferredPrompt.userChoice;

  if (outcome === 'accepted') {
    console.log('[Install] User accepted');
    gtag('event', 'install_accepted');
  } else {
    console.log('[Install] User dismissed');
    gtag('event', 'install_dismissed');
  }

  // Reset — prompt cuma bisa sekali
  deferredPrompt = null;
  installButton.style.display = 'none';
});
```

Track Install Status

```
// Cek apakah PWA sudah terinstall
const isInstalled = window.matchMedia('(display-mode: standalone)').matches
  || window.navigator.standalone === true; // iOS fallback

if (isInstalled) {
  console.log('[PWA] Running as installed app');
  document.body.classList.add('is-installed');
}

// Listen display mode changes
window.matchMedia('(display-mode: standalone)').addEventListener('change', event
  if (event.matches) {
    console.log('[PWA] App launched from home screen');
  }
```

```
});
```

Desktop Install (ChromeOS, Windows, Mac, Linux)

```
// Deteksi platform
function detectPlatform() {
  const ua = navigator.userAgent;
  if (ua.includes('Windows')) return 'windows';
  if (ua.includes('Mac OS')) return 'mac';
  if (ua.includes('Linux')) return 'linux';
  if (ua.includes('CrOS')) return 'chromeos';
  return 'other';
}

// Daftar platform yang support install prompt
const SUPPORTED_PLATFORMS = ['windows', 'mac', 'linux', 'chromeos'];

// Modifikasi UI per platform
if (SUPPORTED_PLATFORMS.includes(detectPlatform())) {
  console.log('[Install] Desktop PWA install supported');
  installButton.textContent = `Install di ${detectPlatform() === 'mac' ? 'Mac' :
}
```

Splash Screen

Splash screen di PWA otomatis dari kombinasi:

- **background_color** → background
- **icons** (192x192) → center icon
- **name** → title text

Customisasi Splash Screen

```
{
  "name": "Notes App",
  "background_color": "#007aff",
  "theme_color": "#007aff",
  "icons": [
    {
      "src": "/icons/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    }
  ]
}
```

Hasil: Splash screen biru dengan icon putih di tengah + tulisan "Notes App"

> **Catatan:** iOS pake meta tag khusus, bukan manifest:
>
> ```
> <link rel="apple-touch-startup-image" href="/splash.png" media="(device-widt
> ```
>
> Tapi cara paling gampang — pake **PWACompat** library biar otomatis.

PWACompat (iOS support)

```
<!-- PWACompat — polyfill buat iOS & browser lama -->
<script async src="https://cdn.jsdelivr.net/npm/pwacompat@2.0.17/pwacompat.min.j
  integrity="sha256-eK8Cz0RhFlbPmJb0v4Wh8N9lsMLe13dZyYmqgUGOl58="
  crossorigin="anonymous"></script>
```

App Shortcuts

Shortcuts muncul pas user **long-press** / **right-click** icon aplikasi.

```
{
  "shortcuts": [
    {
      "name": "Tambah Catatan",
      "short_name": "Tambah",
      "description": "Buka halaman tambah catatan",
      "url": "/new-note",
      "icons": [{ "src": "/icons/shortcut-add.png", "sizes": "96x96" }]
    },
    {
      "name": "Cari Catatan",
      "short_name": "Cari",
      "url": "/search?q=",
      "icons": [{ "src": "/icons/shortcut-search.png", "sizes": "96x96" }]
    }
  ]
}
```

Handle Shortcut di Halaman

```
// Detect shortcut launch
const launchParams = new URLSearchParams(window.location.search);

if (launchParams.get('source') === 'shortcut') {
  console.log('[PWA] Launched from shortcut');
  focusAppropriateSection(launchParams.get('action'));
```

```
}

// Atau pake LaunchQueue API (Chrome)
if ('launchQueue' in window) {
  launchQueue.setConsumer(params => {
    console.log('[PWA] Launch consumer:', params);
    if (params.targetURL?.includes('/new-note')) {
      openNewNoteForm();
    }
  });
}
```

PWA Audit dengan Lighthouse

Cara Menggunakan Lighthouse

1. **Chrome DevTools** → Lighthouse tab → PWA category → Generate report
2. **CLI:** `npx lighthouse https://sitekamu.com --view`
3. **CI/CD:** `npx lighthouse https://sitekamu.com --output json --output-path ./reports/lighthouse.json`

Checklist PWA Audit

Wajib (Score mempengaruhi)

Kriteria	Requirement
□ Register SW	SW terdaftar dengan `install` dan `fetch` event
□ Offline fallback	Halaman offline atau response 200 saat offline
□ Valid manifest	Manifest valid & terpasang
□ HTTPS	Semua resource via HTTPS (kecuali localhost)
□ Precache HTML	Halaman harus di-precache di SW
□ Icons ≥ 192px	Minimal icon 192x192 dan 512x512 di manifest
□ Maskable icon	Minimal satu icon dengan `purpose: maskable`

Opsional (Bonus)

Kriteria	Requirement
□ Splash screen	`background_color` + icon di manifest
□ Theme color	`theme_color` di manifest & `<meta name="theme-color">`

Kriteria	Requirement
□ Content width	Cocok dengan viewport (pakai <meta name="viewport">)
□ Apple touch icon	<link rel="apple-touch-icon">
□ Install prompt	beforeinstallprompt bisa di-trigger
□ Fast loading	First paint < 5 detik di 3G
□ Shortcuts	Minimal 1 shortcut di manifest

Lighthouse Scoring

```
PWA Score:

┌──────────────────────────────────────┐
│ □ Installable          100/100       │
│ □ PWA Optimized        100/100       │
│ ──────────────────────────────────── │
│ Total                    100/100     │
└──────────────────────────────────────┘
```

Otomatis Audit dengan CI

```yaml
# .github/workflows/lighthouse.yml
name: Lighthouse PWA Audit
on: [push]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npm run build

      # Start server di background
      - run: npm run serve & sleep 3

      - name: Run Lighthouse
        uses: treosh/lighthouse-ci-action@v10
        with:
          urls: |
            http://localhost:3000
          uploadArtifacts: true
          temporaryPublicStorage: true
          budgetPath: ./lighthouse-budget.json
```

```
// lighthouse-budget.json
{
  "performance": 80,
  "accessibility": 90,
  "pwa": 100,
  "best-practices": 85
}
```

Full Setup Flow

```
# 1. Buat manifest.json
# 2. Generate icons (pake PWABuilder atau sharp)
# 3. Link manifest di <head>
# 4. Daftarkan Service Worker
# 5. Test install prompt
# 6. Audit dengan Lighthouse
# 7. Fix semua issue
# 8. Deploy
```

Verifikasi Manual

```
// DevTools utility — cek manifest
(async function checkManifest() {
  // 1. Cek manifest link
  const links = document.querySelectorAll('link[rel="manifest"]');
  console.log('[Manifest] Link tags:', links.length);

  // 2. Fetch & display manifest content
  for (const link of links) {
    const response = await fetch(link.href);
    const manifest = await response.json();
    console.table({
      name: manifest.name,
      short_name: manifest.short_name,
      display: manifest.display,
      icons: manifest.icons?.length,
      theme_color: manifest.theme_color,
      start_url: manifest.start_url
    });
  }

  // 3. Check SW registration
  const registrations = await navigator.serviceWorker.getRegistrations();
  console.log('[SW] Registrations:', registrations.length);
```

```
  // 4. Check installability
  if ('onbeforeinstallprompt' in window) {
    console.log('[Install] beforeinstallprompt supported');
  }
})();
```

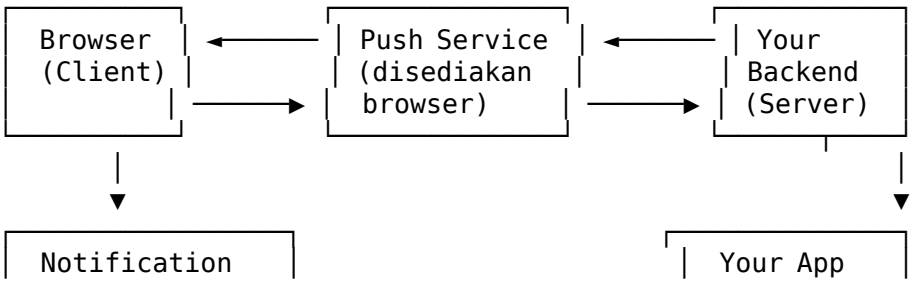
Latihan

1. **Buat manifest.json lengkap** — Isi semua field wajib (name, short_name, start_url, display, icons, theme_color, background_color). Generate icon 192x192 dan 512x512 (pake PWABuilder atau sharp). Link manifest di HTML. Verifikasi di DevTools > Application > Manifest.
2. **Custom install button** — Tampilkan tombol "Install Aplikasi" yang muncul pas `beforeinstallprompt`. Log user choice (accepted/dismissed) ke console. Sembunyikan tombol kalo udah terinstall. Test dengan Lighthouse.
3. **Splash screen + App shortcuts** — Set `background_color` dan `theme_color` di manifest. Tambah 2 shortcuts: "Tambah Catatan" → `/new-note` dan "Cari" → `/search`. Pastikan splash screen muncul pas PWA di-launch dari home screen.
4. **Lighthouse audit & fix** — Jalankan Lighthouse PWA audit. Target score ≥ 90. Dokumentasikan:
   - Yang lolos □
   - Yang gagal □
   - Perbaikan yang dilakukan
   - Screenshot hasil audit

4. Push Notifications

Arsitektur Push Notification

Push notification di PWA melibatkan 4 entitas:



Alur Lengkap

1. **Subscribe** — Client minta izin ke user, dapet PushSubscription object
2. **Kirim ke server** — Client kirim subscription data ke backend
3. **Simpan** — Backend simpan subscription di database
4. **Trigger push** — Backend kirim push message ke Push Service via Web Push Protocol
5. **Deliver** — Push Service kirim ke browser (walau tab tertutup)
6. **Handle event** — Service Worker terima `push` event, tampilin notification
7. **Interaksi** — User klik notification → `notificationclick` event

VAPID Keys

VAPID (Voluntary Application Server Identification) — protokol buat identify server yang ngirim push. Biar Push Service tau siapa yang ngirim.

Generate VAPID Keys

```
# Via CLI — pake web-push npm package
npx web-push generate-vapid-keys

# Output:
# =======================================
# Public Key:
# BParX1x...cYvsbxL8k
# Private Key:
# n2H3q...vP1C8Y
# =======================================
```

Programmatic Generate

```
// scripts/generate-vapid.js
const webpush = require('web-push');

const vapidKeys = webpush.generateVAPIDKeys();
console.log('VAPID Keys generated:\n');
console.log('Public Key:', vapidKeys.publicKey);
console.log('Private Key:', vapidKeys.privateKey);
```

Simpan di Environment Variables

```
# .env
VAPID_PUBLIC_KEY=BParX1x...cYvsbxL8k
VAPID_PRIVATE_KEY=n2H3q...vP1C8Y
VAPID_EMAIL=admin@notesapp.com
```

Subscribe User (Client Side)

Di Halaman Web

```
// push-client.js
const VAPID_PUBLIC_KEY = 'BParX1x...cYvsbxL8k'; // dari server

async function subscribeUser() {
  // 1. Cek support
  if (!('serviceWorker' in navigator) || !('PushManager' in window)) {
    console.log('[Push] Not supported');
    return null;
  }

  // 2. Daftarkan SW (kalo belum)
  const registration = await navigator.serviceWorker.register('/sw.js');
  console.log('[Push] SW registered');

  // 3. Minta izin notifikasi
  const permission = await Notification.requestPermission();
  if (permission !== 'granted') {
    console.log('[Push] Permission denied');
    return null;
  }

  // 4. Subscribe ke push service
  const subscription = await registration.pushManager.subscribe({
    userVisibleOnly: true, // WAJIB — setiap push harus tampil notif
    applicationServerKey: urlBase64ToUint8Array(VAPID_PUBLIC_KEY)
  });

  console.log('[Push] Subscribed:', JSON.stringify(subscription));
  return subscription;
}

// Helper — convert VAPID key (base64 URL safe → Uint8Array)
function urlBase64ToUint8Array(base64String) {
  const padding = '='.repeat((4 - (base64String.length % 4)) % 4);
  const base64 = (base64String + padding)
```

```
    .replace(/\-/g, '+')
    .replace(/_/g, '/');

  const rawData = window.atob(base64);
  const outputArray = new Uint8Array(rawData.length);

  for (let i = 0; i < rawData.length; ++i) {
    outputArray[i] = rawData.charCodeAt(i);
  }
  return outputArray;
}
```

Kirim Subscription ke Server

```
async function saveSubscription(subscription) {
  try {
    const response = await fetch('/api/push/subscribe', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        endpoint: subscription.endpoint,
        keys: {
          p256dh: btoa(String.fromCharCode(...new Uint8Array(
            subscription.getKey('p256dh')
          ))),
          auth: btoa(String.fromCharCode(...new Uint8Array(
            subscription.getKey('auth')
          )))
        },
        userAgent: navigator.userAgent,
        subscribedAt: new Date().toISOString()
      })
    });

    if (response.ok) {
      console.log('[Push] Subscription saved to server');
      return true;
    }
    throw new Error('Failed to save');
  } catch (err) {
    console.error('[Push] Save error:', err);
    return false;
  }
}
```

Cek Status Subscription & Unsubscribe

```
async function checkSubscription() {
  const registration = await navigator.serviceWorker.ready;
  const subscription = await registration.pushManager.getSubscription();

  if (subscription) {
    console.log('[Push] Active subscription:', subscription.endpoint);
    return subscription;
  }

  console.log('[Push] No active subscription');
  return null;
}

async function unsubscribeUser() {
  const registration = await navigator.serviceWorker.ready;
  const subscription = await registration.pushManager.getSubscription();

  if (subscription) {
    const success = await subscription.unsubscribe();
    if (success) {
      console.log('[Push] Unsubscribed');
      // Beritahu server
      await fetch('/api/push/unsubscribe', {
        method: 'POST',
        body: JSON.stringify({ endpoint: subscription.endpoint })
      });
    }
  }
}
```

UI Subscription Toggle

```
// Push toggle button
const pushToggle = document.getElementById('push-toggle');

async function initPushUI() {
  const subscription = await checkSubscription();

  pushToggle.checked = !!subscription;
  pushToggle.addEventListener('change', async event => {
    if (event.target.checked) {
      const sub = await subscribeUser();
      if (sub) await saveSubscription(sub);
    } else {
```

```
      await unsubscribeUser();
    }
  });
}

// Tampilkan status di UI
async function updatePushStatus() {
  const status = document.getElementById('push-status');
  const sub = await checkSubscription();

  if (sub) {
    const permission = Notification.permission;
    status.textContent = `□ Push aktif (${permission})`;
    status.className = 'push-active';
  } else {
    status.textContent = '□ Push tidak aktif';
    status.className = 'push-inactive';
  }
}
```

Service Worker: Handle Push Event

```
// sw.js — handle push event

self.addEventListener('push', event => {
  console.log('[SW Push] Received:', event);

  let data = { title: 'Default Title', body: '', icon: '/icons/icon-192x192.png'

  if (event.data) {
    try {
      const payload = event.data.json();
      data = { ...data, ...payload };
    } catch {
      data.body = event.data.text();
    }
  }

  const options = {
    body: data.body,
    icon: data.icon,
    badge: '/icons/badge-72x72.png',
    image: data.image,
    vibrate: [200, 100, 200],
    data: {
```

```
      url: data.url || '/',
      type: data.type || 'general',
      id: data.id || null,
      dateOfArrival: Date.now()
    },
    tag: data.tag || 'default',
    renotify: data.renotify || false,
    requireInteraction: data.requireInteraction || false,
    silent: data.silent || false,
    actions: data.actions || []
  };

  event.waitUntil(
    self.registration.showNotification(data.title, options)
  );

  // Analytics
  logNotificationReceived(data);
});

function logNotificationReceived(data) {
  // Kirim ke analytics endpoint
  fetch('/api/analytics/push-received', {
    method: 'POST',
    body: JSON.stringify({
      type: data.type,
      timestamp: Date.now()
    }),
    keepalive: true
  }).catch(() => {});
}
```

Notification Payload dari Server

```
// Server → Push Service → Browser
const pushPayload = {
  title: 'Catatan Baru Tersimpan!',
  body: 'Catatan "Meeting Notes" berhasil di-sync ke cloud.',
  icon: '/icons/icon-192x192.png',
  badge: '/icons/badge-72x72.png',
  image: '/images/sync-complete.png',
  tag: 'sync-notes',
  renotify: false,
  requireInteraction: true,
  silent: false,
```

```
  vibrate: [200, 100, 200, 100, 100],
  data: {
    url: '/notes/123',
    type: 'sync',
    id: '123'
  },
  actions: [
    {
      action: 'open',
      title: 'Buka Catatan'
    },
    {
      action: 'dismiss',
      title: 'Tutup'
    }
  ],
  timestamp: Date.now()
};
```

Handle Notification Click

```
// sw.js — handle notification click

self.addEventListener('notificationclick', event => {
  console.log('[SW] Notification click:', event);

  const notification = event.notification;
  const action = event.action;
  const data = notification.data || {};

  notification.close();

  let responsePromise;

  if (action === 'dismiss') {
    // User dismiss — gak perlu action
    responsePromise = Promise.resolve();
  } else if (action === 'open' || !action) {
    // Buka URL yang ditentukan
    responsePromise = handleOpenAction(data);
  } else if (action === 'reply') {
    // Buka halaman dengan reply mode
    responsePromise = handleReplyAction(data);
  } else if (action === 'snooze') {
    // Snooze notification — kirim reminder nanti
```

```
    responsePromise = handleSnoozeAction(data);
  } else {
    // Custom action
    responsePromise = handleCustomAction(action, data);
  }

  event.waitUntil(responsePromise);
});

async function handleOpenAction(data) {
  const urlToOpen = data.url || '/';

  const clients = await self.clients.matchAll({
    type: 'window',
    includeUncontrolled: true
  });

  // Cek apakah udah ada tab yang terbuka
  for (const client of clients) {
    if (client.url.includes(urlToOpen) && 'focus' in client) {
      return client.focus();
    }
  }

  // Kalo gak ada — buka tab baru
  if (clients.length === 0) {
    return self.clients.openWindow(urlToOpen);
  }

  // Fokus ke tab pertama
  return clients[0].focus();
}

async function handleReplyAction(data) {
  const replyUrl = `/notes/${data.id}/reply`;
  return self.clients.openWindow(replyUrl);
}

async function handleSnoozeAction(data) {
  // Register periodic sync untuk remind 30 menit lagi
  const registration = await self.registration;
  try {
    await registration.periodicSync.register('remind-' + data.id, {
      minInterval: 30 * 60 * 1000 // 30 menit
    });
  } catch {
```

```
    // Fallback: kirim notif lagi dalam 30 detik (simulasi)
    setTimeout(() => {
      self.registration.showNotification('Reminder: ' + (data.reminderTitle || '
        body: 'Klik untuk buka.',
        tag: 'reminder-' + data.id
      });
    }, 30000);
  }
}

function handleCustomAction(action, data) {
  // Handler buat custom actions
  console.log('[SW] Custom action:', action, data);
  return self.clients.openWindow(`/${action}?ref=${data.id}`);
}
```

Notification Close Event

```
self.addEventListener('notificationclose', event => {
  console.log('[SW] Notification dismissed without click:', event.notification.t

  // Bisa kirim analytics
  event.waitUntil(
    fetch('/api/analytics/push-dismissed', {
      method: 'POST',
      body: JSON.stringify({
        tag: event.notification.tag,
        timestamp: Date.now()
      })
    }).catch(() => {})
  );
});
```

Sending Push dari Server

Setup web-push (Node.js)

```
npm install web-push
```

```
// server/push.js
const webpush = require('web-push');
require('dotenv').config();

// Konfigurasi VAPID
webpush.setVapidDetails(
  `mailto:${process.env.VAPID_EMAIL}`,
```

```
  process.env.VAPID_PUBLIC_KEY,
  process.env.VAPID_PRIVATE_KEY
);

// Database subscriptions (simulasi array — ganti pake DB beneran)
let subscriptions = [];

// API: Subscribe
async function handleSubscribe(req, res) {
  const { endpoint, keys, userAgent } = req.body;

  // Validasi
  if (!endpoint || !keys?.p256dh || !keys?.auth) {
    return res.status(400).json({ error: 'Invalid subscription' });
  }

  // Simpan subscription
  subscriptions.push({
    endpoint,
    keys,
    userAgent,
    subscribedAt: new Date().toISOString(),
    active: true
  });

  console.log('[Push] New subscriber:', endpoint.slice(-20));
  res.json({ success: true });
}

// API: Unsubscribe
async function handleUnsubscribe(req, res) {
  const { endpoint } = req.body;
  subscriptions = subscriptions.filter(s => s.endpoint !== endpoint);
  console.log('[Push] Unsubscribed:', endpoint.slice(-20));
  res.json({ success: true });
}

// API: Get subscriptions (admin)
async function getSubscriptions(req, res) {
  res.json({
    total: subscriptions.length,
    subscriptions: subscriptions.map(s => ({
      endpoint: s.endpoint.slice(-20) + '...',
      userAgent: s.userAgent,
      subscribedAt: s.subscribedAt
    }))
```

```
  });
}

// Fungsi kirim push ke semua subscriber
async function sendPushToAll(payload) {
  const results = [];

  for (const sub of subscriptions) {
    try {
      await webpush.sendNotification(
        {
          endpoint: sub.endpoint,
          keys: {
            p256dh: sub.keys.p256dh,
            auth: sub.keys.auth
          }
        },
        JSON.stringify(payload),
        {
          TTL: 24 * 60 * 60, // 24 jam — kalo offline, di-retry
          urgency: 'normal'   // 'low' | 'normal' | 'high'
        }
      );
      results.push({ endpoint: sub.endpoint.slice(-20), status: 'sent' });
    } catch (err) {
      if (err.statusCode === 410 || err.statusCode === 404) {
        // Subscription expired / invalid — hapus
        console.log('[Push] Removing invalid subscription');
        subscriptions = subscriptions.filter(s => s.endpoint !== sub.endpoint);
        results.push({ endpoint: sub.endpoint.slice(-20), status: 'removed' });
      } else {
        console.error('[Push] Send error:', err);
        results.push({ endpoint: sub.endpoint.slice(-20), status: 'failed', erro
      }
    }
  }

  return results;
}

// API: Trigger push broadcast
async function handleSendPush(req, res) {
  const { title, body, url, type, tag } = req.body;

  const payload = {
    title: title || 'Pemberitahuan',
```

```
    body: body || '',
    icon: '/icons/icon-192x192.png',
    badge: '/icons/badge-72x72.png',
    tag: tag || 'broadcast',
    requireInteraction: false,
    data: {
      url: url || '/',
      type: type || 'broadcast'
    },
    actions: [
      { action: 'open', title: 'Buka' },
      { action: 'dismiss', title: 'Tutup' }
    ]
  };

  const results = await sendPushToAll(payload);
  res.json({
    sent: results.filter(r => r.status === 'sent').length,
    removed: results.filter(r => r.status === 'removed').length,
    failed: results.filter(r => r.status === 'failed').length,
    details: results
  });
}

// API: Send push to specific user
async function sendPushToUser(endpoint, payload) {
  const sub = subscriptions.find(s => s.endpoint === endpoint);
  if (!sub) throw new Error('Subscription not found');

  await webpush.sendNotification(
    {
      endpoint: sub.endpoint,
      keys: {
        p256dh: sub.keys.p256dh,
        auth: sub.keys.auth
      }
    },
    JSON.stringify(payload)
  );
}

// Express Router
const express = require('express');
const router = express.Router();

router.post('/subscribe', handleSubscribe);
```

```
router.post('/unsubscribe', handleUnsubscribe);
router.get('/subscriptions', getSubscriptions);
router.post('/send', handleSendPush);
router.post('/send/:endpoint', async (req, res) => {
  try {
    await sendPushToUser(req.params.endpoint, req.body);
    res.json({ success: true });
  } catch (err) {
    res.status(404).json({ error: err.message });
  }
});

module.exports = router;
```

Express Server Setup

```
// server/index.js
const express = require('express');
const pushRouter = require('./push');

const app = express();
app.use(express.json());
app.use('/api/push', pushRouter);

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`[Server] Running on port ${PORT}`);
  console.log(`[Server] VAPID configured: ${!!process.env.VAPID_PUBLIC_KEY}`);
});
```

Notification Actions & Interaktivitas

Action Types

Action	Ikon Deskriptif	Fungsi
`open`	Buka	Navigasi ke URL
`dismiss`	Tutup	Tutup notifikasi
`reply`	Balas	Buka halaman reply
`snooze`	Tunda	Tunda 30 menit
`archive`	Arsip	Arsipkan (API call)
`mark-read`	Baca	Tandai sudah dibaca
`delete`	Hapus	Hapus entri
`custom`	⋯	Action kustom

Rich Notification

```
// sw.js — rich notification dengan image & actions
function showRichNotification(data) {
  const options = {
    body: data.body,
    icon: '/icons/icon-192x192.png',
    badge: '/icons/badge-72x72.png',
    image: data.image || '/images/default-notif.png',
    vibrate: [200, 100, 200],
    silent: false,
    requireInteraction: true,
    tag: data.tag,

    // Data yang di-pass ke click handler
    data: {
      url: data.url,
      type: data.type,
      metadata: data.metadata
    },

    // Action buttons
    actions: [
      {
        action: 'open',
        title: '□ Buka',
        icon: '/icons/action-view.png'
      },
      {
        action: 'archive',
        title: '□ Arsipkan',
        icon: '/icons/action-archive.png'
      },
      {
        action: 'snooze',
        title: '□ Tunda 30m',
        icon: '/icons/action-snooze.png'
      },
      {
        action: 'dismiss',
        title: '✖ Tutup'
      }
    ]
  };

  return self.registration.showNotification(data.title, options);
```

```
}
```

Inline Reply (Chrome Desktop)

```
// sw.js — inline reply action
self.addEventListener('notificationclick', event => {
  const notification = event.notification;
  const action = event.action;
  const data = notification.data;

  if (action === 'reply') {
    // Chrome 123+ support inline reply
    // event.reply akan berisi text dari user
    const replyText = event.reply || '';

    if (replyText) {
      // Kirim reply ke server
      fetch('/api/notes/' + data.id + '/reply', {
        method: 'POST',
        body: JSON.stringify({ text: replyText }),
        keepalive: true
      });
    }
  }

  notification.close();
});
```

Notification dengan Progress Bar

```
// Simulasi progress — update notif berulang
async function showProgressNotification() {
  let progress = 0;

  const notification = await self.registration.showNotification('Sync Progress',
    body: `0% — Memulai sinkronisasi...`,
    tag: 'sync-progress',
    silent: true,
    requireInteraction: true
  });

  const interval = setInterval(async () => {
    progress += 10;

    await self.registration.showNotification('Sync Progress', {
```

```
      body: `${progress}% — Sinkronisasi...`,
      tag: 'sync-progress',
      silent: true,
      renotify: true  // Biar notifnya kedengeran
    });

    if (progress >= 100) {
      clearInterval(interval);
      await self.registration.showNotification('Sync Selesai □', {
        body: 'Semua catatan tersinkronisasi!',
        tag: 'sync-progress',
        icon: '/icons/icon-192x192.png'
      });
    }
  }, 1000);
}
```

Notification Analytics

Track di Client

```
// sw.js — push analytics
self.addEventListener('push', event => {
  // ... show notification ...

  const analyticsData = {
    action: 'push_received',
    type: data.type || 'unknown',
    tag: data.tag || 'default',
    timestamp: Date.now(),
    clientTime: new Date().toISOString()
  };

  event.waitUntil(
    fetch('/api/analytics/push', {
      method: 'POST',
      body: JSON.stringify(analyticsData),
      headers: { 'Content-Type': 'application/json' },
      keepalive: true  // Biar request tetap dikirim walau SW dimatikan
    }).catch(() => {})
  );
});

self.addEventListener('notificationclick', event => {
  const analyticsData = {
    action: 'notification_click',
```

```
    tag: event.notification.tag,
    actionName: event.action || 'default',
    timestamp: Date.now()
  };

  event.waitUntil(
    fetch('/api/analytics/push-click', {
      method: 'POST',
      body: JSON.stringify(analyticsData),
      keepalive: true
    }).catch(() => {})
  );
});
```

Dashboard Analytics (Server)

```
// server/analytics.js
const analytics = {
  totalSent: 0,
  totalReceived: new Set(),  // Set of endpoints
  totalClicks: new Set(),
  actions: {},
  daily: {}
};

function trackSent(payload) {
  analytics.totalSent++;
  const day = new Date().toISOString().slice(0, 10);
  analytics.daily[day] = analytics.daily[day] || { sent: 0, received: 0, clicks:
  analytics.daily[day].sent++;
}

function trackReceived(endpoint) {
  analytics.totalReceived.add(endpoint);
  const day = new Date().toISOString().slice(0, 10);
  analytics.daily[day] = analytics.daily[day] || { sent: 0, received: 0, clicks:
  analytics.daily[day].received++;
}

function trackClick(endpoint, action) {
  analytics.totalClicks.add(endpoint);
  analytics.actions[action] = (analytics.actions[action] || 0) + 1;
  const day = new Date().toISOString().slice(0, 10);
  analytics.daily[day] = analytics.daily[day] || { sent: 0, received: 0, clicks:
  analytics.daily[day].clicks++;
```

```
}

function getAnalytics() {
  return {
    totalSent: analytics.totalSent,
    totalReceived: analytics.totalReceived.size,
    totalClicks: analytics.totalClicks.size,
    ctr: analytics.totalSent > 0
      ? ((analytics.totalClicks.size / analytics.totalSent) * 100).toFixed(2) +
      : '0%',
    actionsByType: analytics.actions,
    daily: analytics.daily
  };
}

app.get('/api/analytics/push-summary', (req, res) => {
  res.json(getAnalytics());
});
```

Full Push Flow Implementation

```
// === CLIENT SIDE ===
// push-client.js — init di halaman utama
import { subscribeUser, saveSubscription, checkSubscription } from './push-clien

async function initPush() {
  const sub = await checkSubscription();
  if (!sub) {
    // Tawarkan subscribe pas user melakukan action
    document.getElementById('enable-push').addEventListener('click', async () =>
      const newSub = await subscribeUser();
      if (newSub) {
        await saveSubscription(newSub);
        alert('Notifikasi aktif!');
      }
    });
  }
}

initPush();

// === SERVICE WORKER ===
// sw.js — lengkap
self.addEventListener('push', event => {
  const data = event.data?.json() || {};
```

```
  const options = {
    body: data.body || '',
    icon: '/icons/icon-192x192.png',
    badge: '/icons/badge-72x72.png',
    data: { url: data.url || '/', type: data.type },
    actions: data.actions || []
  };
  event.waitUntil(self.registration.showNotification(data.title || 'Notes App',
});

self.addEventListener('notificationclick', event => {
  console.log('[SW] Click:', event.action);
  event.notification.close();

  if (event.action !== 'dismiss') {
    const url = event.notification.data?.url || '/';
    event.waitUntil(self.clients.openWindow(url));
  }
});

// === SERVER SIDE ===
// Kirim push via API
fetch('/api/push/send', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    title: 'Catatan Baru',
    body: 'Budi nambahin catatan "Meeting Notes"',
    url: '/notes/123',
    type: 'share',
    tag: 'share-notes'
  })
});
```

Latihan

1. **Subscribe & save subscription** — Generate VAPID keys pake `npx web-push generate-vapid-keys`. Implementasi subscribe flow di client: minta izin, subscribe, kirim subscription ke server. Verifikasi subscription muncul di server console.
2. **Kirim push dari server** — Setup Express server dengan web-push. Implementasi endpoint POST `/api/push/send` yang kirim notifikasi ke semua subscriber. Payload: title, body, icon, data.url. Test pake curl atau Postman.

3. **Notification actions + click handling** — Tambah 2 action button di notifikasi: “Buka Catatan” dan “Arsipkan”. Di `notificationclick`, handle masing-masing action. Buka URL kalo “Buka”, kirim DELETE request kalo “Arsipkan”. Log analytics ke console.
4. **Unsubscribe & cleanup** — Implementasi tombol “Nonaktifkan Notifikasi” di UI. Handle unsubscribe (client + server). Di server, cleanup subscription yang expired (status 410). Tampilkan status notifikasi (aktif/nonaktif) di halaman.